

PYTHON

1. What are the primitive data types in python? string, integer, float, boolean

non-primitive datatypes (set, tuple, list, dictionary)

2. python lambda function? A lambda function in Python is a small anonymous function defined using the lambda keyword. It can have any number of arguments but only one expression, and it returns the result of the expression when called.

using normal lambda

```
# Basic lambda function to add two numbers
```

```
add = lambda x, y: x + y
```

```
result = add(5, 3)
```

```
print(result) # Output: 8
```

```
# Using lambda with map to square each number in a list
```

```
numbers = [1, 2, 3, 4, 5]
```

```
squared_numbers = list(map(lambda x: x ** 2, numbers))
```

```
print(squared_numbers) # Output: [1, 4, 9, 16, 25]
```

3. what is append vs extend in python? append adds a single element to the end of the list.

extend adds each element from an iterable to the end of the list.

```
# Example of append
```

```
my_list = [1, 2, 3]
```

```
my_list.append([4, 5]) # Appending a list
```

```
print(my_list) # Output: [1, 2, 3, [4, 5]]
```

4. what is generator? A generator in Python is a special type of iterable that produces values one at a time and only when needed, using the yield statement. This makes generators memory efficient because they don't store the entire sequence in memory at once.

Let's say we are working on a large data processing project, where we need to read a huge log file containing millions of entries. Instead of loading the entire file into memory, which can lead to performance issues and high memory usage, we can use a generator to process the file line by line.

5. what is decorator? Decorators (wrapper) are a way to modify or enhance functions or methods. They are often used for logging, enforcing access control, instrumentation, or caching. A decorator is a function that takes another function as an argument and returns a new function.

6. what is in-it? "In my understanding, __init__ is a special method used to initialize newly created objects. It's known as the initializer or constructor. When you create an instance of a class, Python automatically calls the __init__ method to set up the object with initial values.

Scenario 2: School Management System

Answer: "we worked on a student analysis program where we needed to track student information effectively. To achieve this, I created a Student class. The __init__ method was crucial because it allowed me to initialize each student's name, age, and grade upon creation. This made it straightforward for the administration team to input new students and manage their records efficiently, promoting an organized way to handle academic data."

7. what is lead and lag? "LEAD retrieves the value from a subsequent row, useful for comparing current values with future values.

LAG retrieves the value from a previous row, useful for comparing current values with past values."

Both functions use a specified offset and can return a default value if no corresponding row exists. They are useful for time series analysis and tracking changes over sequential data.

Scenario 2:

In a student progress analysis project, I used both LEAD and LAG to evaluate student behaviour over time. For example, I am using LAG to compare a student engagement level in the current month with their engagement level from the previous month. Simultaneously, I used LEAD to predict their engagement for the following month based on their current behaviour. This dual approach provided a comprehensive view of student trends, helping our academic team devise targeted campaigns to improve student marks based on both historical data and future expectations

8. how did you optimize the python code need a interview question? I used several strategies to optimize my Python code:

Efficient Algorithms: I chose the right algorithms and data structures to improve performance, like using dictionaries for quick lookups.

Profiling: I used tools like cProfile to find slow parts of the code and focused on optimizing those.

* List Comprehensions: I replaced loops with list comprehensions for cleaner and faster code.

* Built-in Functions: I used Python's built-in functions because they are optimized for performance.

Concurrency: For tasks that could run at the same time, I used threading or multiprocessing to speed things up.

* Caching: I used caching to store results of expensive function calls and avoid repeating calculations.

* Database Optimization: When working with databases, I optimized queries by using indexes and selecting only necessary data.

SQL

1. order of execution? FROM: Identify the tables involved and join them if necessary.

WHERE: Filter the rows based on the specified conditions.

GROUP BY: Group the filtered rows based on specified columns.

HAVING: Filter the grouped rows based on conditions.

SELECT: Determine which columns to include in the final result set.

ORDER BY: Sort the result set based on specified columns.

LIMIT/OFFSET:

2. Joins? joins are used to combine rows from two or more tables based on a related column between them. Joins allow you to retrieve data that spans multiple tables. The most common types of joins include INNER JOIN , LEFT JOIN , RIGHT JOIN , FULL JOIN , CROSS JOIN

3. query optimization technique in sql? I optimize SQL queries using several techniques

* Indexing

* Selecting Specific Columns using SELECT .

* Using WHERE Clauses

* Using JOINS

4. diff btwn where and having? WHERE Clause Used to filter records before any groupings are made and its Works with individual rows in a table.

HAVING Clause Used to filter records after groupings are made, Works with aggregated data (like results of GROUP BY).

5. data structure of sql? In SQL, data structures include tables, which consist of rows and columns representing entities and their attributes. Indexes improve data retrieval speed, while views provide virtual tables based on queries. Relationships, defined by foreign keys, establish connections between tables, enhancing data organization and integrity.

6. most common error u faced in joints ? Ambiguous error : An ambiguous error in SQL happens when two tables have columns with the same name, and the database doesn't know which one to use. To fix this, always use the table name or an alias before the column name, like `table_name.column_name`.

7. group by and partition by in sql? GROUP BY: Reduces the result set by collapsing rows into single rows per group, usually used with aggregate functions.

PARTITION BY: Keeps all rows in the result set and allows calculations across a specified partition of data using window functions.

8. Indexing - column based? In SQL, indexing refers to the process of creating a data structure (an index) that improves the speed of data retrieval operations on a database table. An index is similar to an index in a book, allowing the database to find rows more quickly without scanning the entire table.

9. rank and dense rank ? Window Functions

`row_number()`: Assigns a unique sequential integer to rows within a partition.

`rank()`: Provides the rank of each row within a partition.

`dense_rank()`: Similar to rank but does not leave gaps in ranking.

AWS

1. diff btwn lambda and spark? AWS Lambda is ideal for small datasets, event-driven tasks and real-time processing, especially when integrating closely with AWS services.

Apache Spark is suited for handling large datasets, complex transformations, and analytics, providing powerful tools for big data processing.

2. lambda? AWS Lambda is a serverless computing service that enables us to run code in response to events without managing servers. It automatically scales applications by executing code in response to triggers from AWS services like S3, DynamoDB, and API Gateway.

3. s3? Amazon S3 (Simple Storage Service) is a scalable object storage service provided by AWS. It's designed to store and retrieve any amount of data from anywhere on the web

4. cloudwatch? Amazon CloudWatch is a monitoring and observability service designed for AWS resources and applications. It helps you track metrics, collect log files, set alarms, and automate actions based on your monitoring data

5. crawler? Purpose: The crawler scans data in your data stores (like Amazon S3, Amazon RDS, etc.) to infer the schema and create or update tables in the AWS Glue Data Catalog.

Schema Detection: It automatically detects the structure of your data (like data types and partitions) and can handle various data formats, including CSV, JSON, Parquet, and more.

Data Catalog: The crawler populates the Data Catalog, which serves as a central repository for your metadata, making it easier to find and access data.

6. athena? Amazon Athena is an interactive query service that allows you to analyze data directly in Amazon S3 using standard SQL. It's serverless, so you don't need to set up or manage any infrastructure.

7. glue? AWS Glue simplifies the ETL process and integrates well with the broader AWS ecosystem, making it a powerful tool for data engineers and analysts looking to manage and prepare large datasets efficiently.

8. diff btwn glue and lambda? AWS Glue is ideal for complex ETL processes and data preparation tasks, while AWS Lambda is suited for running small pieces of code in response to events. They can complement each other in data workflows, but they serve different purposes within the AWS ecosystem.

9. sns? Amazon Simple Notification Service (SNS) is a fully managed messaging service provided by AWS. SNS is a powerful and flexible messaging service that simplifies the process of building distributed systems, allowing for efficient communication and event-driven architectures.

10. cron scheduler? A cron scheduler is a time-based job scheduler in Unix-like operating systems. It allows users to schedule scripts or commands to run at specified intervals, such as daily, weekly, monthly, or at specific times

11. AWS services you know? EC2, S3, Lambda, Glue, Managed Airflow, CloudWatch, SNS, SSM, IAM, Athena